

TP3 - Audit, cartographie et traitement des données avec SQL

Page de titre

- **Nom et prénom (du binôme)** : [] compl[ter]
- **Titre du TP** : TP3 - Audit, cartographie et traitement des données avec SQL
- **Date de réalisation** : 2024-02-08
- **Enseignant / cours** : [] compl[ter]

Description du travail réalisé

Le travail consiste à construire un mini-pipeline ETL multi-sources pour nettoyer, normaliser, fusionner et préparer des données issues de quatre fichiers CSV ('source_ventes.csv', 'source_produits.csv', 'source_commandes.csv', 'source_articles.csv'). Les étapes principales sont :

1. **Extraction** : création des tables de staging et importation des fichiers CSV.
2. **Transformation** : nettoyage des valeurs invalides (dates, prix négatifs, stocks nuls), harmonisation des formats, suppression des doublons et uniformisation des chaînes.
3. **Fusion et enrichissement** : fusion des produits/articles vers une dimension unique et enrichissement des ventes avec des champs calculés.
4. **Chargement** : création des tables finales pour l'analyse ('ventes_analyse', 'commandes_analyse', 'dim_produits').
5. **Journalisation** : traçabilité des chargements via une table de logs.
6. **Optimisation** : indexation et partitionnement.

Contexte des données

- 'source_ventes.csv' : ventes quotidiennes (ID_VENTE, DATE_VENTE, ID_PRODUIT, CLIENT, QUANTITE, PRIX_UNITAIRE, VILLE).
- 'source_produits.csv' : catalogue produits (ID_PRODUIT, NOM_PRODUIT, CATEGORIE, PRIX_CATALOGUE, STOCK).
- 'source_commandes.csv' : commandes clients (ID_COMMANDE, DATE_COMMANDE, ID_ARTICLE, CLIENT, QUANTITE, PRIX_UNITAIRE, VILLE, MODE_PAIEMENT, CODE_POSTAL, ETAT_COMMANDE).
- 'source_articles.csv' : articles associés (ID_ARTICLE, NOM_ARTICLE, CATEGORIE, PRIX_CATALOGUE, STOCK, FOURNISSEUR, DATE_AJOUT, ACTIF, TAUX_TVA).

Requêtes effectuées (avec commentaires)

> Les requêtes ci-dessous peuvent être exécutées dans MySQL. Chaque requête est accompagnée d'une explication.

1) Création des tables de staging

```
CREATE TABLE staging_ventes (  
  ID_VENTE INT,  
  DATE_VENTE VARCHAR(20),  
  ID_PRODUIT VARCHAR(10),  
  CLIENT VARCHAR(100),  
  QUANTITE INT,  
  PRIX_UNITAIRE VARCHAR(20),  
  VILLE VARCHAR(100)
```

```
);
```

- **Fonction** : stocker les ventes brutes sans contrainte stricte de type.
- **Raison** : éviter les erreurs d'import liées aux formats invalides.
- **Résultat attendu** : table prête pour le nettoyage.

```
CREATE TABLE staging_produits (  
  ID_PRODUIT INT,  
  NOM_PRODUIT VARCHAR(100),  
  CATEGORIE VARCHAR(50),  
  PRIX_CATALOGUE DECIMAL(10,2),  
  STOCK INT
```

```
ID_PRODUIT VARCHAR(10),
NOM_PRODUIT VARCHAR(200),
CATEGORIE VARCHAR(100),
PRIX_CATALOGUE VARCHAR(20),
STOCK VARCHAR(20)
);
- **Fonction** : accueillir le catalogue brut.
- **Raison** : isoler les erreurs (NaN, None, prix nogatifs).
- **Rosultat attendu** : table staging complote.
```

```
CREATE TABLE staging_commandes (
  ID_COMMANDE VARCHAR(10),
  DATE_COMMANDE VARCHAR(20),
  ID_ARTICLE VARCHAR(10),
  CLIENT VARCHAR(100),
  QUANTITE INT,
  PRIX_UNITAIRE VARCHAR(20),
  VILLE VARCHAR(100),
  MODE_PAIEMENT VARCHAR(50),
  CODE_POSTAL VARCHAR(10),
  ETAT_COMMANDE VARCHAR(50)
);
- **Fonction** : stocker les commandes brutes.
- **Raison** : permettre un nettoyage progressif.
- **Rosultat attendu** : table staging prote au traitement.
```

```
CREATE TABLE staging_articles (
  ID_ARTICLE VARCHAR(10),
  NOM_ARTICLE VARCHAR(200),
  CATEGORIE VARCHAR(100),
  PRIX_CATALOGUE VARCHAR(20),
  STOCK VARCHAR(20),
  FOURNISSEUR VARCHAR(100),
  DATE_AJOUT VARCHAR(20),
  ACTIF VARCHAR(10),
  TAUX_TVA VARCHAR(10)
);
- **Fonction** : importer les articles bruts.
- **Raison** : conserver la trace des anomalies avant correction.
- **Rosultat attendu** : table staging complote.
```

2) Import des CSV

```
LOAD DATA INFILE '/path/source_ventes.csv'
INTO TABLE staging_ventes
FIELDS TERMINATED BY ','
IGNORE 1 LINES;
- **Fonction** : charger les donnoes des ventes.
- **Raison** : ingestion rapide et traotable.
- **Rosultat attendu** : lignes importoes en staging.
```

(R^op^oter pour les autres fichiers en adaptant la table cible.)

3) Nettoyage des dates et des valeurs invalides

```
UPDATE staging_ventes
```

```

SET DATE_VENTE = NULL
WHERE DATE_VENTE IS NULL
  OR DATE_VENTE = ''
  OR DATE_VENTE NOT REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2}$';
- **Fonction** : rep rer les dates invalides.
- **Raison** :  viter les conversions en erreur.
- **R sultat attendu** : dates invalides remplac es par NULL.

```

```

UPDATE staging_ventes
SET PRIX_UNITAIRE = NULL
WHERE PRIX_UNITAIRE IN ('NaN', 'None', '')
  OR CAST(PRIX_UNITAIRE AS DECIMAL(10,2)) <= 0;
- **Fonction** : neutraliser les prix invalides ou n gatifs.
- **Raison** : pr server la coh rence financi re.
- **R sultat attendu** : prix invalides mis   NULL.

```

4) Uniformisation des textes

```

UPDATE staging_ventes
SET CLIENT = UPPER(CLIENT),
  VILLE = UPPER(VILLE);
- **Fonction** : homog n iser les libell s.
- **Raison** :  viter des doublons logiques lors des regroupements.
- **R sultat attendu** : valeurs en majuscules.

```

5) Fusion produits + articles (dimension)

```

CREATE TABLE dim_produits AS
SELECT
  p.ID_PRODUIT,
  COALESCE(p.NOM_PRODUIT, a.NOM_ARTICLE) AS NOM_PRODUIT,
  COALESCE(p.CATEGORIE, a.CATEGORIE) AS CATEGORIE,
  CAST(NULLIF(p.PRIX_CATALOGUE, 'NaN') AS DECIMAL(10,2)) AS PRIX_CATALOGUE,
  CAST(NULLIF(p.STOCK, 'NaN') AS SIGNED) AS STOCK,
  a.FOURNISSEUR,
  a.DATE_AJOUT,
  a.ACTIF,
  a.TAUX_TVA
FROM staging_produits p
LEFT JOIN staging_articles a
  ON p.ID_PRODUIT = a.ID_ARTICLE;
- **Fonction** : cr er une dimension produit fusionn e.
- **Raison** : obtenir une vue unique des r f rences produits/articles.
- **R sultat attendu** : table 'dim_produits' pr te pour l  analyse.

```

6) Fusion des ventes avec la dimension

```

CREATE TABLE ventes_analyse AS
SELECT
  v.ID_VENTE,
  v.DATE_VENTE,
  v.ID_PRODUIT,
  v.CLIENT,
  v.QUANTITE,
  CAST(v.PRIX_UNITAIRE AS DECIMAL(10,2)) AS PRIX_UNITAIRE,
  v.VILLE,
  (v.QUANTITE * CAST(v.PRIX_UNITAIRE AS DECIMAL(10,2))) AS TOTAL_VENTE,

```

```
d.NOM_PRODUIT,  
d.CATEGORIE
```

```
FROM staging_ventes v  
LEFT JOIN dim_produits d
```

```
ON v.ID_PRODUIT = d.ID_PRODUIT;
```

- ****Fonction**** : créer une table de faits enrichie.
- ****Raison**** : analyser les ventes par produit/catégorie.
- ****Résultat attendu**** : table 'ventes_analyse' avec 'TOTAL_VENTE'.

7) Journalisation ETL

```
CREATE TABLE log_etl (  
ID_LOG INT AUTO_INCREMENT PRIMARY KEY,  
ETAPE VARCHAR(100),  
NB_LIGNES INT,  
DATE_EXECUTION DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

- ****Fonction**** : traçabilité des traitements.
- ****Raison**** : contrôle de qualité et audit.
- ****Résultat attendu**** : table de logs centralisée.

```
INSERT INTO log_etl (ETAPE, NB_LIGNES)  
VALUES ('Chargement staging_ventes', (SELECT COUNT(*) FROM staging_ventes));
```

- ****Fonction**** : tracer le nombre de lignes chargées.
- ****Raison**** : vérifier l'extraction.
- ****Résultat attendu**** : log enregistré.

8) Indexation

```
CREATE INDEX idx_ventes_date ON ventes_analyse (DATE_VENTE);  
CREATE INDEX idx_ventes_produit ON ventes_analyse (ID_PRODUIT);  
CREATE INDEX idx_ventes_ville ON ventes_analyse (VILLE);
```

- ****Fonction**** : accélérer les requêtes analytiques.
- ****Raison**** : optimiser les filtres fréquents.
- ****Résultat attendu**** : index disponibles.

Captures de données (à insérer)

1. ****Import CSV dans la table staging**** : capture montrant l'exécution de 'LOAD DATA INFILE'.
2. ****Exemple de nettoyage**** : capture d'une requête 'SELECT' après nettoyage des dates/prix.
3. ****Table dim_produits**** : capture du résultat de la fusion produits/articles.
4. ****Table ventes_analyse**** : capture avec le champ 'TOTAL_VENTE'.
5. ****Table log_etl**** : capture montrant l'insertion de logs.

Chaque capture devra être commentée dans le document final.